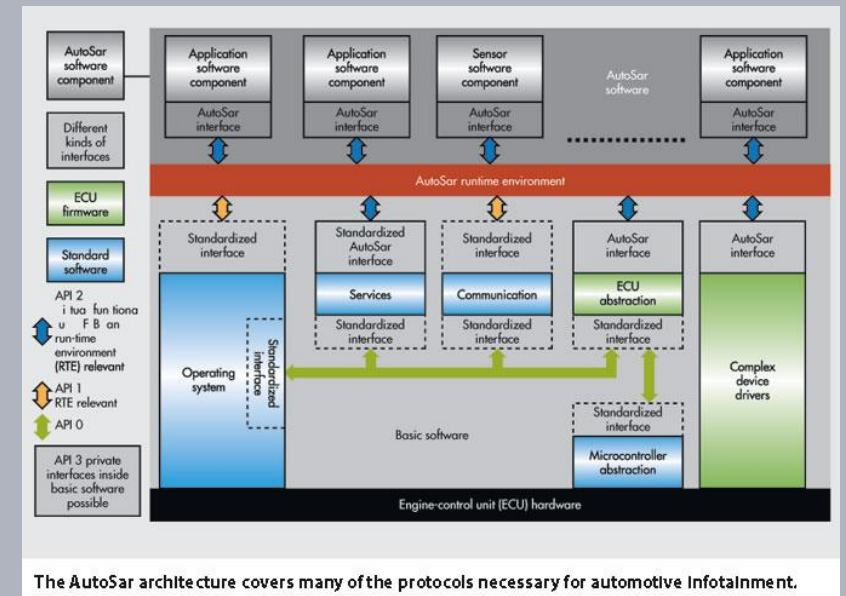


Advanced Microcontroller Architectures and Embedded OS

OSEK, Erika on PSOC continued

Prof. Dr.-Ing. Peter Fromm

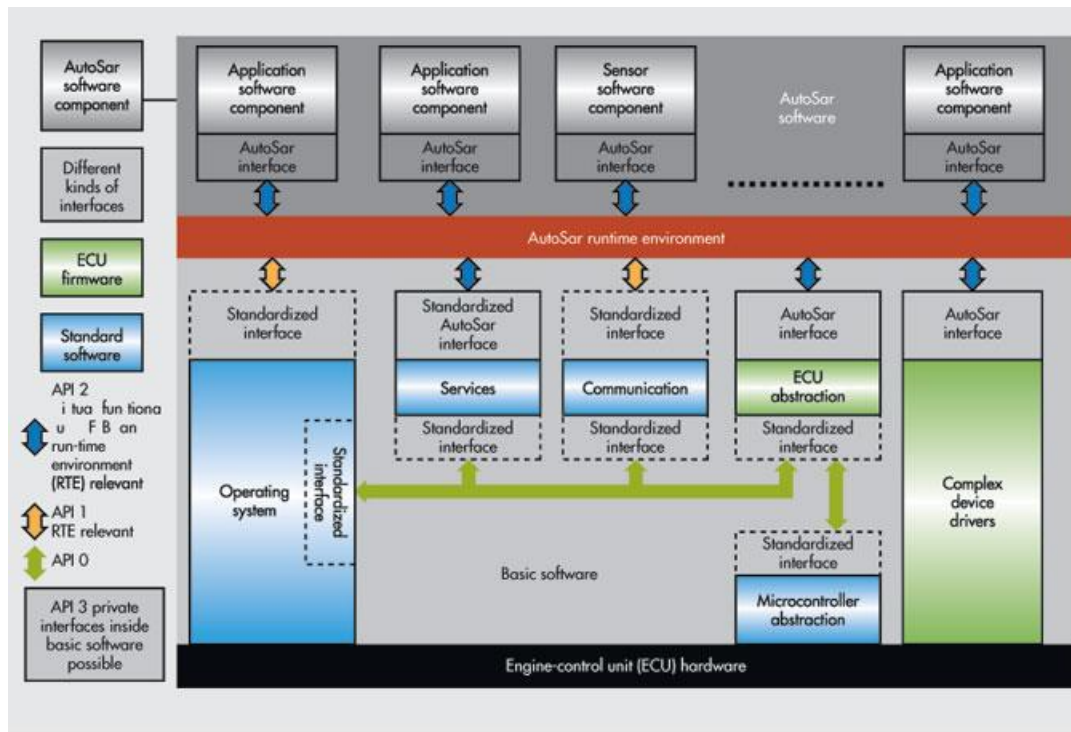


The AutoSar architecture covers many of the protocols necessary for automotive Infotainment.

Content

- PSOC project structure and configuration
- Re-using project frames
- Integrating Doxygen in the Build Process
- Which tasks
- Choosing Task Priorities
- Abstracting events and timing services
- Erika Interrupts
- Estimating Stacksize
- Finishing the blinker example

PSOC Project Structure



The AutoSar architecture covers many of the protocols necessary for automotive Infotainment.

Folder asw contains application software

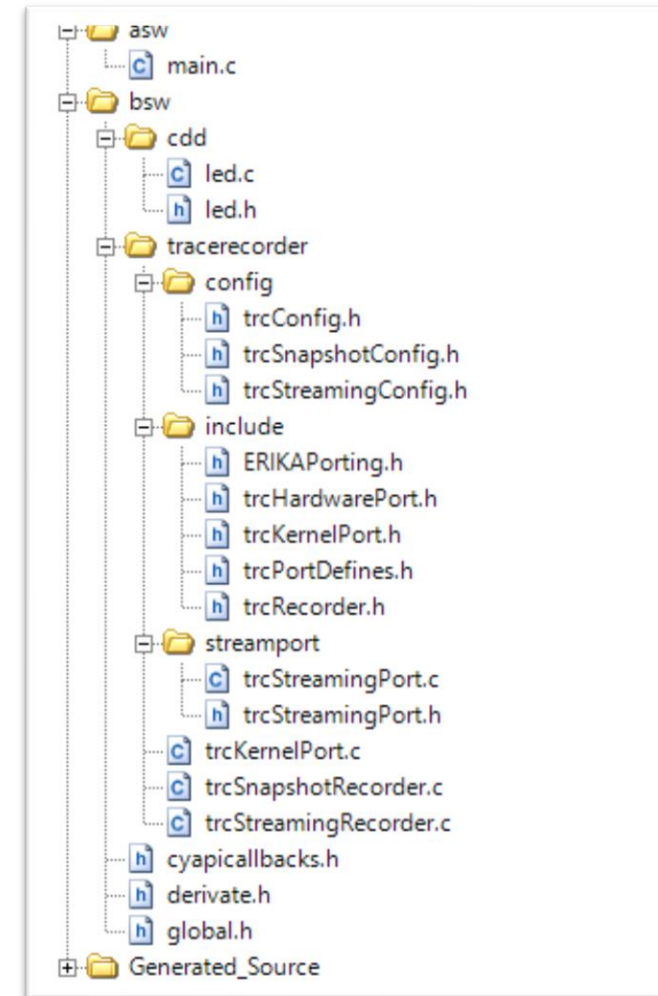
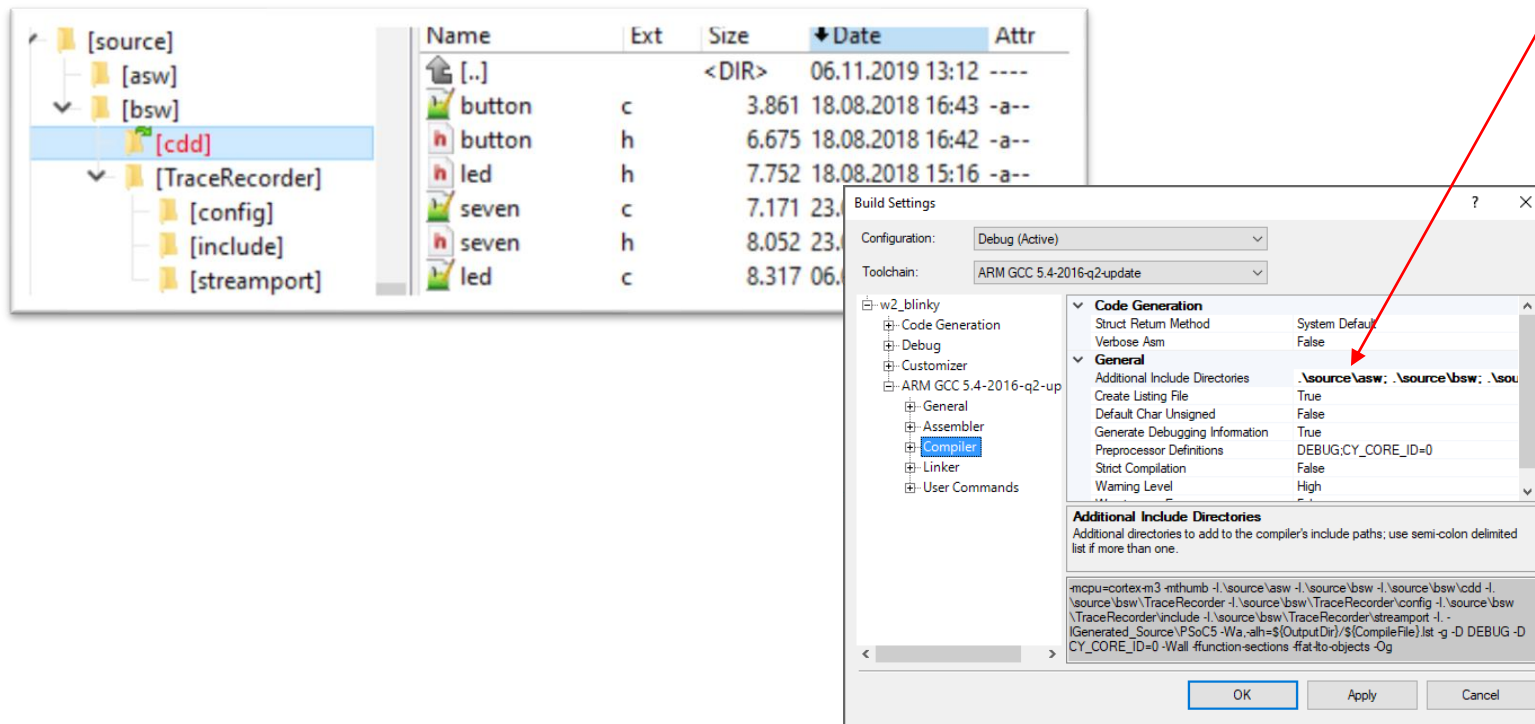
Folder bsw contains basic software, e.g. complex driver

mcal code is generated by PSOC creator

Concept of “logical folders”

Other than Eclipse, the folder structure must be kept in sync manually at three places:

1. Folder structure on the hard disc
2. Logical folder structure in PSOC Creator
3. Include structure in the compiler settings



Global.h and Derivate.h

These files provide a (== our) low level abstraction of the machine and OS architecture.

- Type definitions (uint8_t,...)
- Service definitions (delay jobs,...)

```
//----- [Datatypes and related definitions]

typedef signed   char      sint8_t;          /**< \brief      -128 .. +127          */
typedef unsigned char      uint8_t;         /**< \brief      0 .. 255             */
typedef signed   short     sint16_t;        /**< \brief     -32768 .. +32767       */
typedef unsigned short     uint16_t;        /**< \brief      0 .. 65535          */
typedef signed   long      sint32_t;        /**< \brief    -2147483648 .. +2147483647 */
#ifndef uint32_t
    typedef unsigned long    uint32_t;      /**< \brief      0 .. 4294967295      */
#endif
typedef float     float32_t;                /**< \brief  single precision floating point number (4 byte) */
typedef double    float64_t;                /**< \brief  double precision floating point number (8 byte) */
typedef signed   long long   sint64_t;      /**< \brief -9223372036854775808 .. +9223372036854775807 */
typedef unsigned long long   uint64_t;      /**< \brief      0 .. 18446744073709551615 */
typedef unsigned char      boolean_t;       /**< \brief  for use with TRUE/FALSE */
typedef char         char_t;               /**< \brief  Character Datatype*/

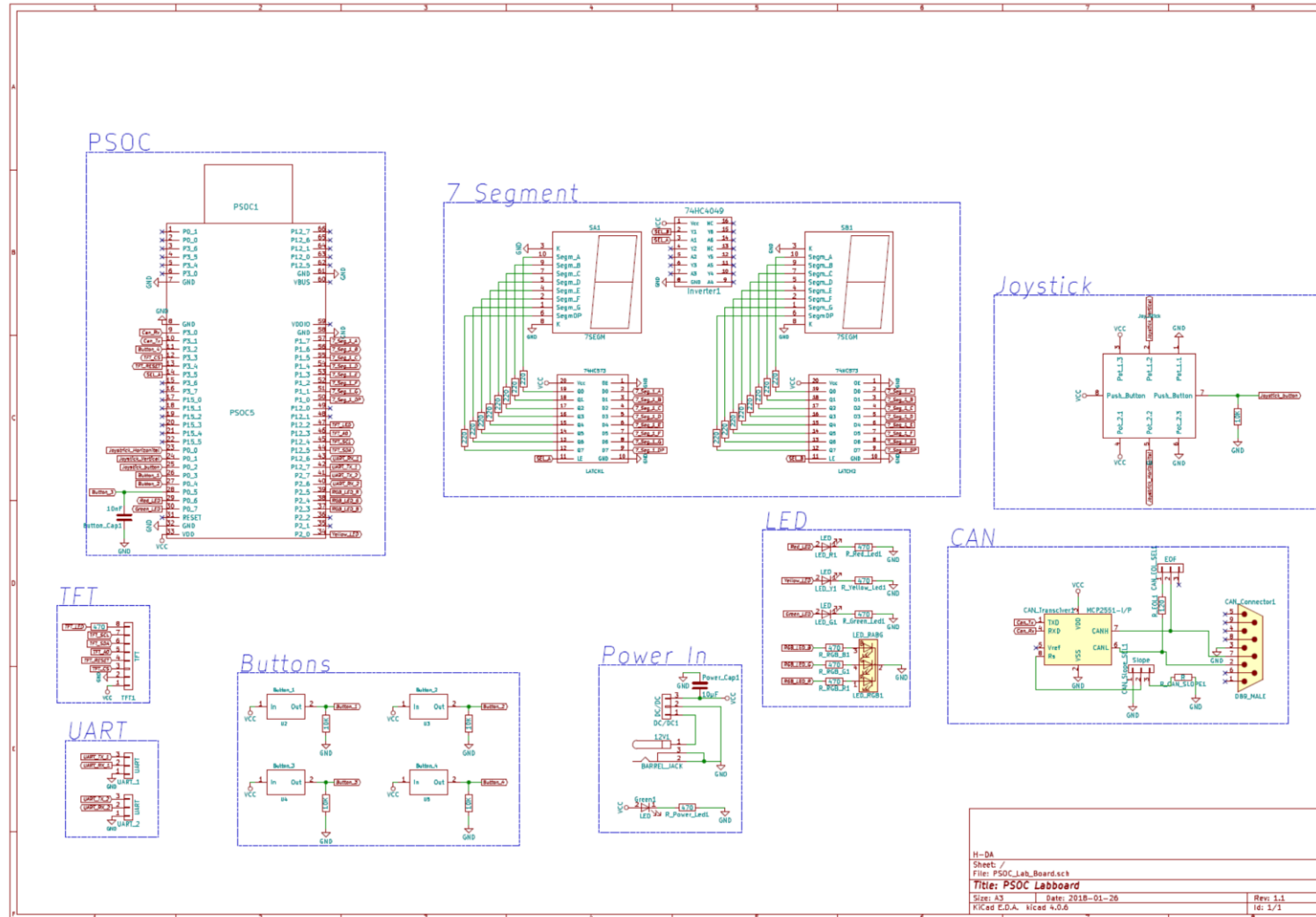
#define uint8_t_MAX      0xFF               /**< \brief  maximum value of uint8_t*/
#define uint16_t_MAX     0xFFFF            /**< \brief  maximum value of uint16_t*/
#define uint32_t_MAX     0xFFFFFFFF        /**< \brief  maximum value of uint32_t*/

#ifndef TRUE
    #define TRUE         ((boolean_t) 1==1) /**< \brief  Value is true (boolean_t type) */
#endif

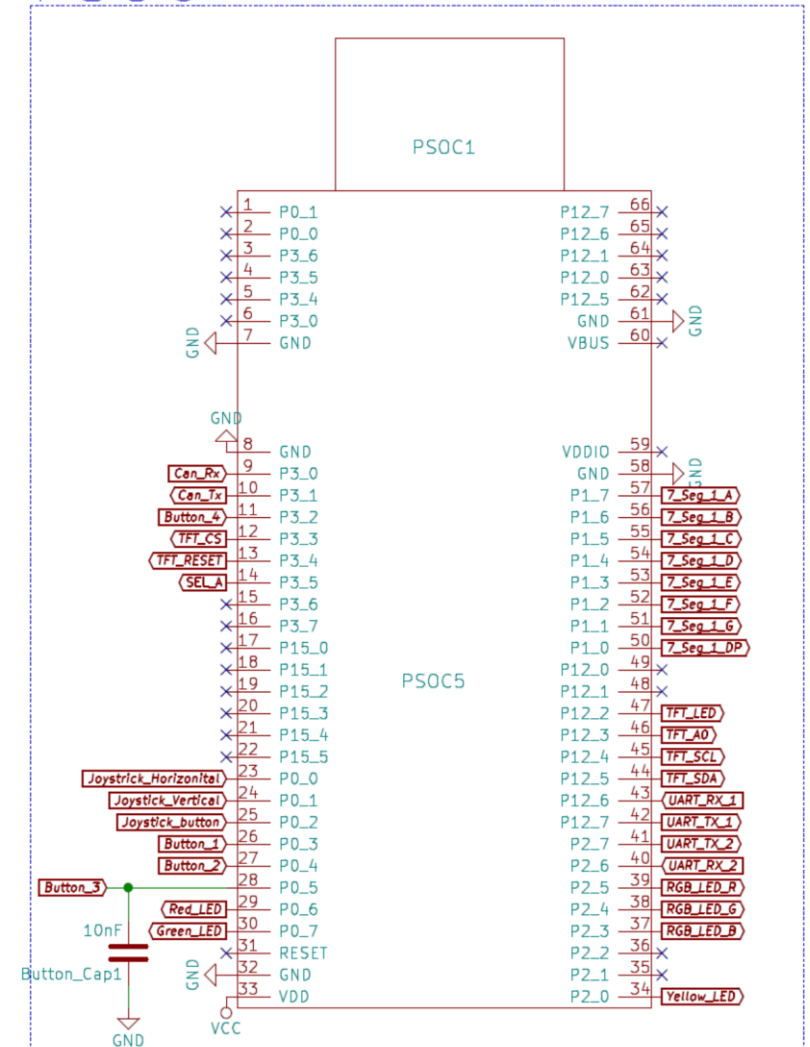
#ifndef FALSE
    #define FALSE        ((boolean_t) 1==0) /**< \brief  Value is true (boolean_t type) */

```

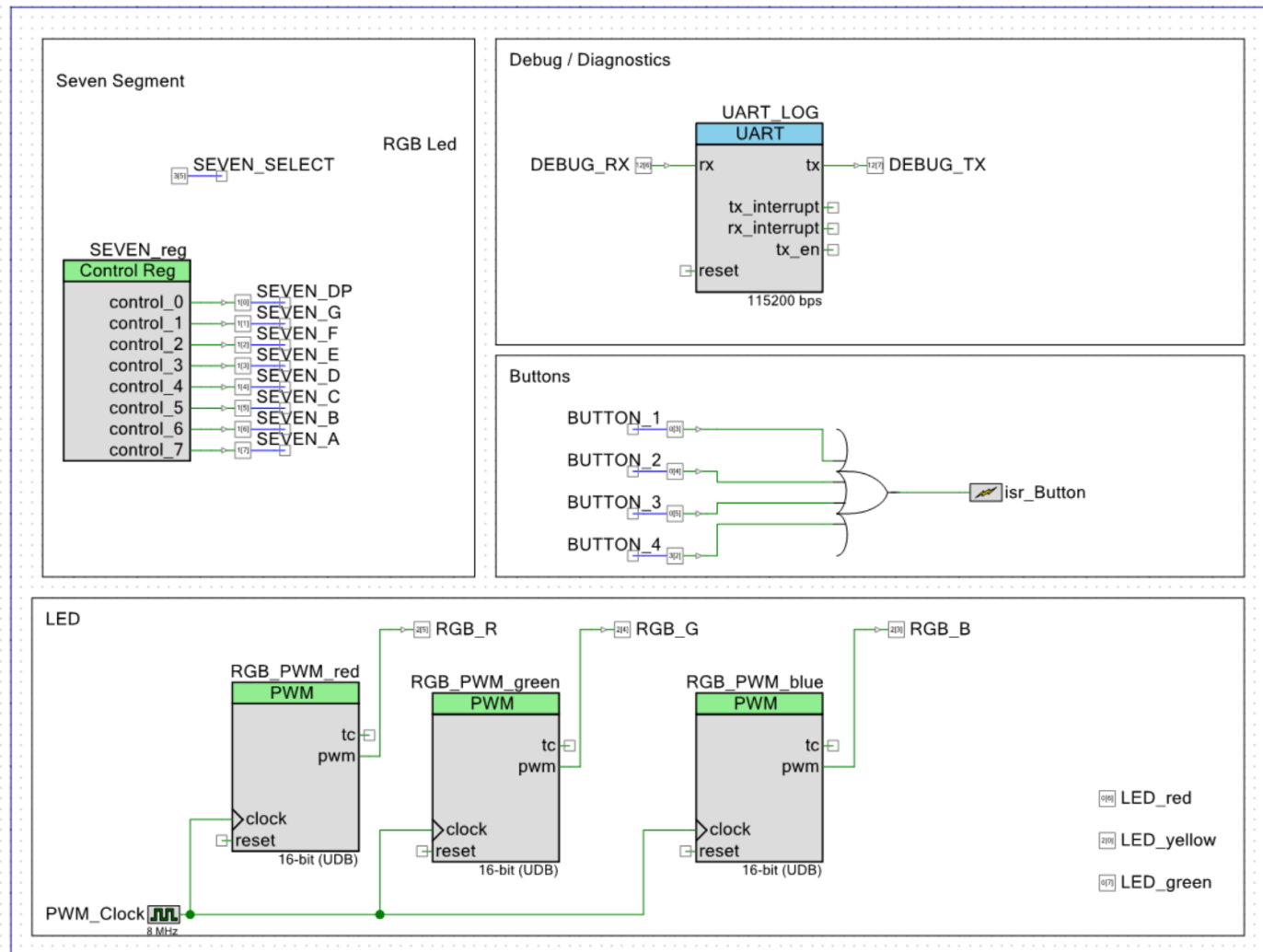
Creating the peripheral configuration



PSOC



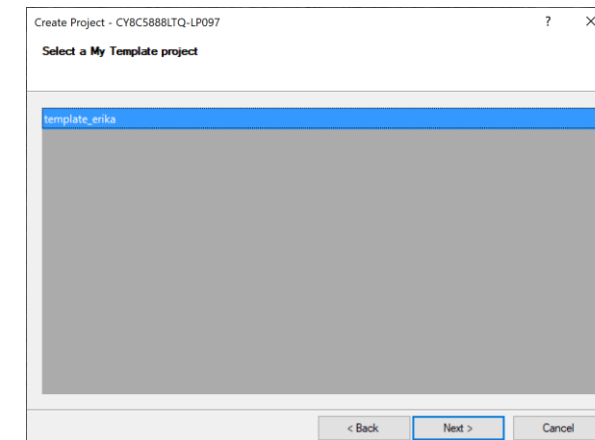
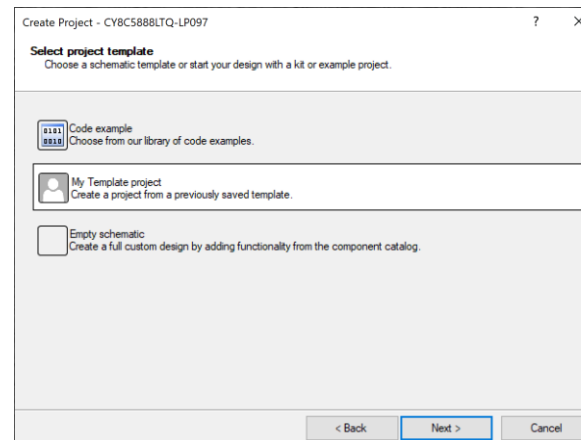
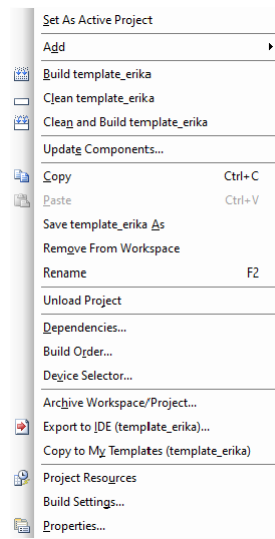
PSOC Design



■	BUTTON_1	P0[3]
■	BUTTON_2	P0[4]
■	BUTTON_3	P0[5]
■	BUTTON_4	P3[2]
■	DEBUG_RX	P12[6]
■	DEBUG_TX	P12[7]
■	LED_green	P0[7]
■	LED_red	P0[6]
■	LED_yellow	P2[0]
■	RGB_B	P2[3]
■	RGB_G	P2[4]
■	RGB_R	P2[5]
■	SEVEN_A	P1[7]
■	SEVEN_B	P1[6]
■	SEVEN_C	P1[5]
■	SEVEN_D	P1[4]
■	SEVEN_DP	P1[0]
■	SEVEN_E	P1[3]
■	SEVEN_F	P1[2]
■	SEVEN_G	P1[1]
■	SEVEN_SELECT	P3[5]

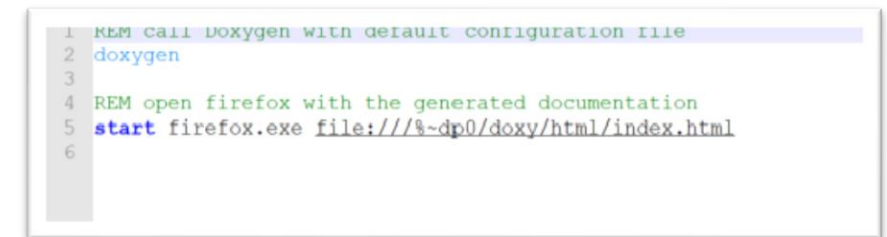
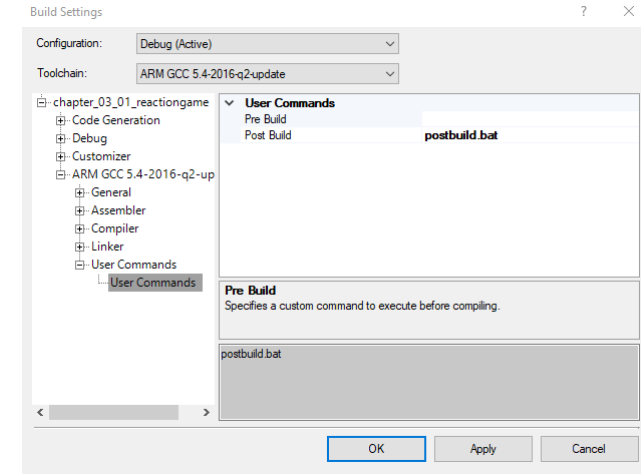
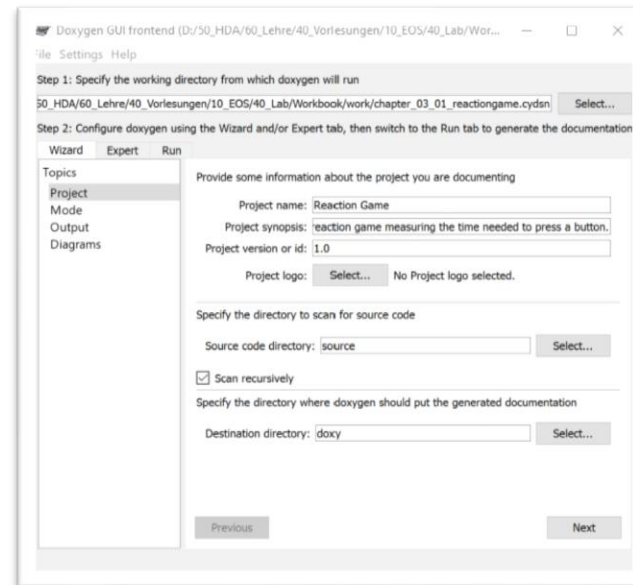
Re-using project frames

- Create a project with a default OS configuration (e.g. init and background task)
- Provide a definition for all peripherals
- Compile and test the framework
- Add it to your template projects
- Alternative: Copy and Paste Project – requires manual configuration of security setting / not very stable!



Integrating Doxygen (and other tools)

- You can add post-build commands to the make process
- Using Doxywizard, create a doxygen file for your project
- Upon every build, a new version of the documentation is generated automatically



Let's create a first real project structure...



Example: Turn Indicator

Task: Development of the control unit for a turn indicator

Requirements:

- If the left switch is pressed, the left indicator will blink with a frequency of 1Hz
- If the right switch is pressed, the right indicator will blink with a frequency of 1Hz
- If the emergency switch is activated, both indicators will blink
- Tipping the left/right switch will let the blinker blink for 5s

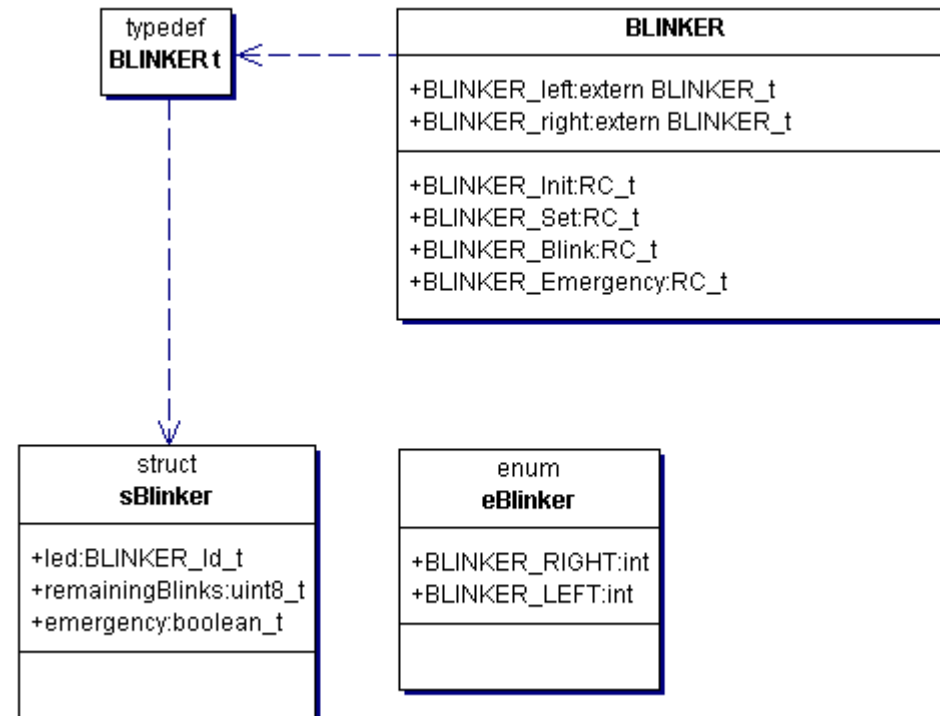


Analysing Requirements

Object Oriented Analysis and Design

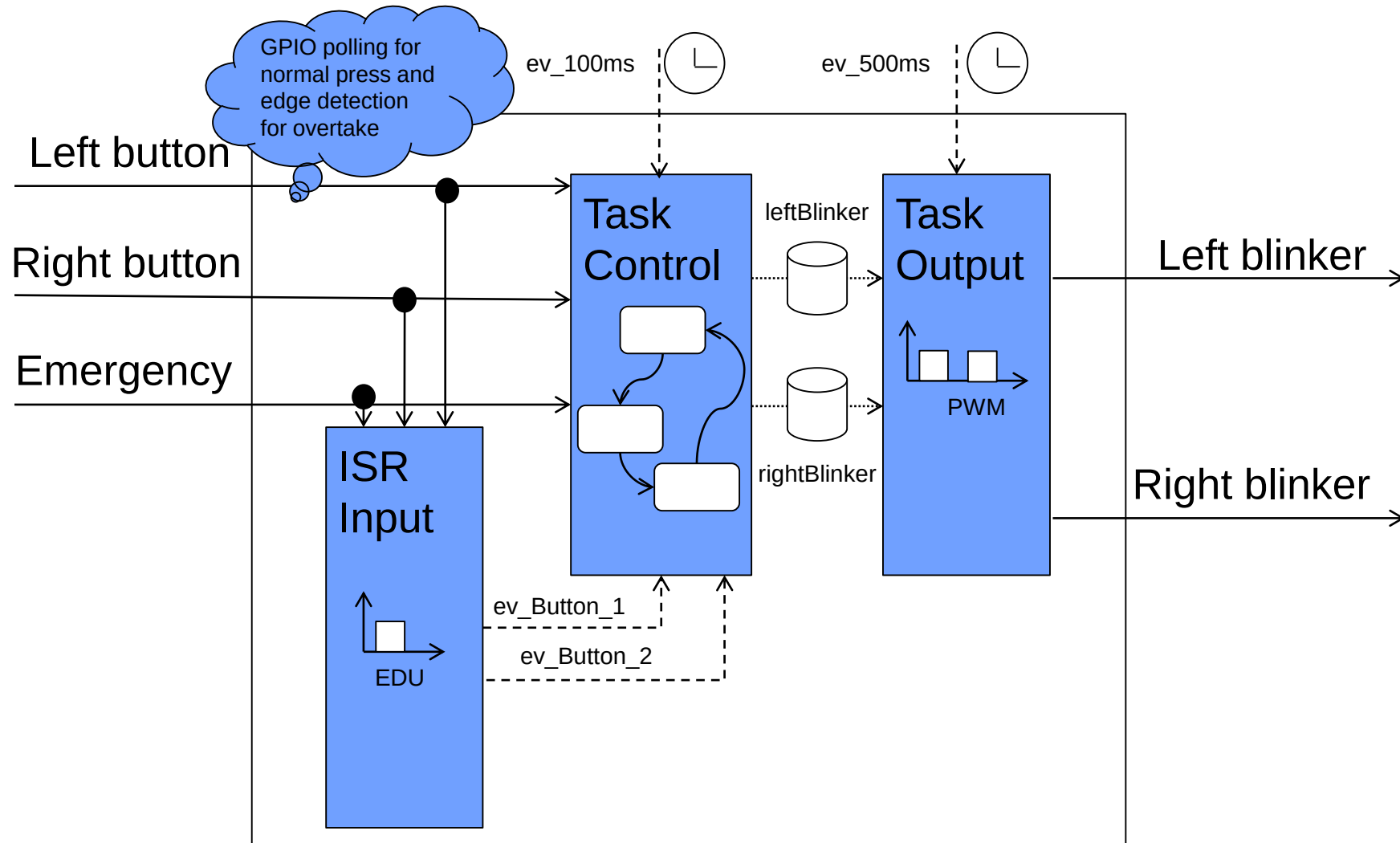
How shall we describe the blinker and design the blinker API?

- We have two blinker objects
- Which can be on or off
- Besides turning them explicitly on or off, they also have a kind of a count down logic for the overtake button
- In the case of an emergency button press they should remain on

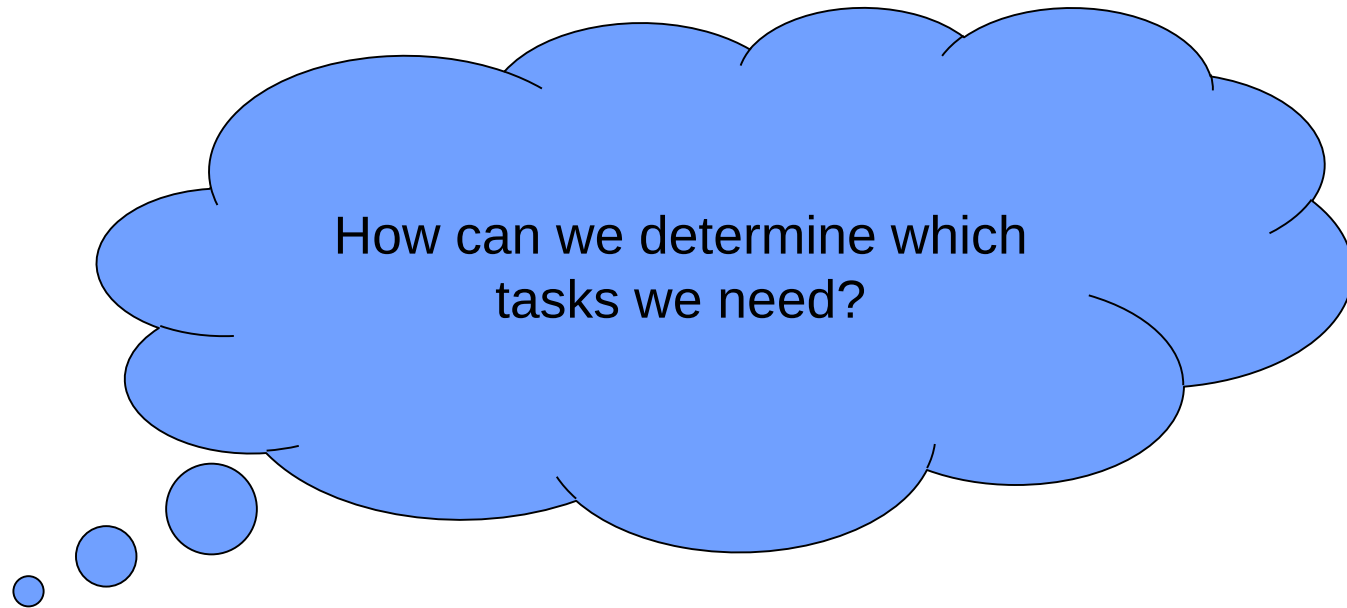


Analysing Requirements

Tasks, Triggers and Signalflow



Key Question - Which Tasks?



Some design rules...

Rule	Justification
Use as few tasks as possible	• The simplest system is running in one task. No races, no preemption....
Add tasks for independent (time-related) execution	• If jobs are executed in parallel, they will be placed in individual tasks. • If a job is woken up by some event or trigger, it should be placed in a task
Put slow preemptable jobs in an own task	• Longrunners (e.g. driving the TFT display) are tampering the realtime behaviour of a system • They should be placed in a low priority task
Distribute activation time	• Use the Alarm offset to distribute the execution time over the slots
Use tasks for resource protection (freedom from interference)	• More advanced operating systems use tasks to create resource pools (e.g. by using the MPU) • Putting safety functions in such sandboxes protects them against malware

Some design rules...

Rule	Justification
Choose the right ticktime	• Fast ticktime (<1ms) allows a precise timebase • But creates a certain overhead (many OS scheduling points) • The ticktime must be fast enough to run the fastest process. Recommended 1ms • Very fast processes should be implemented using timer ISR's
Consider lightweight alternatives	• Runnables instead of tasks • Callbacks instead of tasks
Avoid preemption by design	• As long as it makes sense • Any preemption introduces the risk of races

Events

- Events in OSEK are implemented as global events, i.e. every event will have an own bitmask
- This limits the number of events to 32 (uint32_t)

Idea:

- Have an application event which will take an enum as payload and fires a single OS event
- Benefit: You get 0xFFFFFFFF different events but only 1 OS event is being used
- Additional Benefit: You get a real FIFO buffer (or priority queue, or...)

```
/**
 * Initialise a message object and allocate memory for the buffer
 * \param MESSAGE_t* const me      - [IN/OUT] message queue object
 * \param EventMaskType event      - [IN] event which will be fired in case of new message
 * \param TaskType task           - [IN] receiver task for event
 * \param uint16_t size           - [IN] size of buffer
 * @return RC_SUCCESS or error code
 */
RC_t MSG_init(MESSAGE_t* const me, EventMaskType event, TaskType task, uint16_t size)
{
    //Communication
    me->m_event = event;
    me->m_receiverTask = task;

    //Buffer
    me->m_bufferSize = size;
    me->m_bufferFillLevel = 0;
    me->m_bufferReadIndex = 0;
    me->m_bufferWriteIndex = 0;

    me->m_Buffer = (MESSAGE_content_t*)malloc(me->m_bufferSize * sizeof(MESSAGE_content_t));
    if (0 == me->m_Buffer) return RC_ERROR_MEMORY;

    for (uint16_t i = 0; i < me->m_bufferSize; i++)
    {
        me->m_Buffer[i] = 0;
    }

    return RC_SUCCESS;
}
```

```
/**
 * Initialise a message object and allocate memory for the buffer
 * \param MESSAGE_t* const me      - [IN/OUT] message queue object
 * \param MESSAGE_content_t data    - [IN] data to be written
 * @return RC_SUCCESS or error code
 */
RC_t MSG_write(MESSAGE_t* const me, MESSAGE_content_t data)
{
    if (me->m_bufferFillLevel < me->m_bufferSize)
    {
        me->m_Buffer[me->m_bufferWriteIndex++] = data;
        me->m_bufferWriteIndex %= me->m_bufferSize;
        me->m_bufferFillLevel++;

        SetEvent(me->m_receiverTask, me->m_event);

        return RC_SUCCESS;
    }
    else
    {
        return RC_ERROR_BUFFER_FULL;
    }
}
```

Timer ticks

- Although alarms are comparable cheap, they still occupy OS resources.
- Similar to events, also timing triggers can be implemented in an own class.
- Extending the event mechanism introduced before

```
/**
 * Initialise a timer object and set the receiver of the timeout message
 * \param TIMER_t* const me          - [IN/OUT] timer object
 * \param MESSAGE_t* const* pMessage - [IN] Message which will receive the notification
 * \param MESSAGE_content_t ev       - [IN] event which will be fired
 *
 * @return RC_SUCCESS or error code
 */
RC_t TIMER_init(TIMER_t* const me, MESSAGE_t* const pMessage, MESSAGE_content_t ev)
{
    me->pMessage = pMessage;
    me->ev = ev;
    me->current_ms = 0;
    me->timeout_ms = 0;
    me->mode = TIMER_OFF;

    return RC_SUCCESS;
}
```

```
/**
 * Will tick the timer and fire the event in case a timeout has occurred.
 * In cyclic mode, the timer will be reloaded
 * \param TIMER_t* const me          - [IN/OUT] timer object
 * \param TIMER_tick_t ticks         - [IN] ticks which have elapsed
 * @return RC_SUCCESS or error code
 */
RC_t TIMER_tick(TIMER_t* const me, TIMER_tick_t ticks)
{
    //Check if timer is active
    if (TIMER_OFF == me->mode) return RC_ERROR_INVALID_STATE;

    me->current_ms += ticks;

    //Check if timeout has been reached
    if (me->current_ms >= me->timeout_ms)
    {
        //Notify receiver
        MSG_write(me->pMessage, me->ev);

        if (TIMER_CYCLIC == me->mode)
        {
            //Rerun timer
            me->current_ms = 0;
        }
        else if (TIMER_SINGLESHOT == me->mode)
        {
            TIMER_stop(me);
        }
        else
        {
            return RC_ERROR_NOT_IMPLEMENTED;
        }
    }

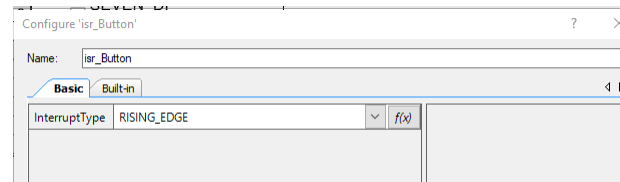
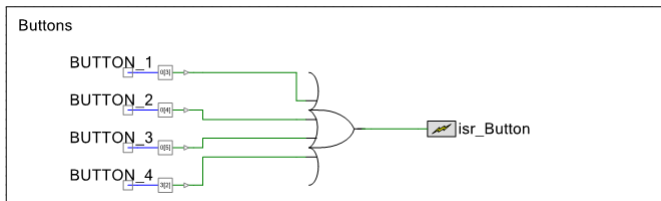
    return RC_SUCCESS;
}
```

Choosing task priorities

- In general, tasks of high importance should have a higher priority as compared to less important tasks
- Tasks processing input typically have a higher priority over tasks processing logic or output (to avoid loss of data on the incoming side)
- Tasks running (safety) critical code have a higher priority
- Tasks having a short runtime may have a higher priority
- Note: Lot's of pre-emption's have a negative impact on the system performance
- When several tasks are activated at the same schedule point, the one having the highest priority will start.
- The alarm offset parameter can be used to distribute load over time

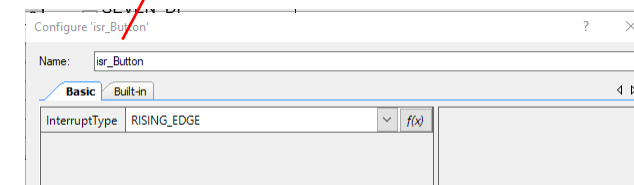
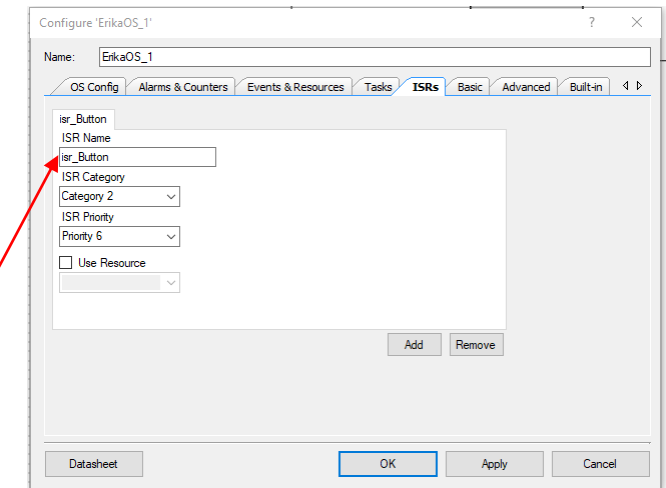
The missing part : Interrupts

- Interrupts in Erika / PSOC are a bit nasty, as two generators are competing
- As a consequence, internal interrupts should preferably not be applied when using Erika, use explicit interrupt objects instead
- Step 1 – create a hardware interrupt



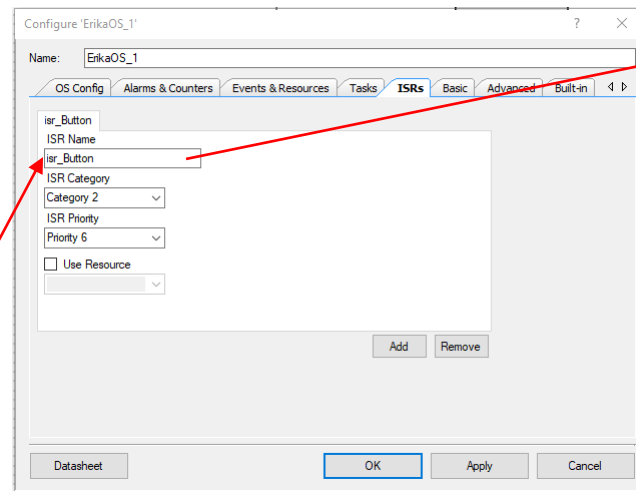
Step 2 – Add the ISR to Erika

- Note: The name of the ISR object and the ERIKA ISR must be identical!
- Category 1 ISR are bare metal ISR's without OS awareness
- Category 2 ISR's are required, if OS calls like SetEvent will be applied in the ISR
- ISR and Task priorities are inverted: Low interrupt priority number means high priority.
- ISR's automatically have a higher priority than tasks.

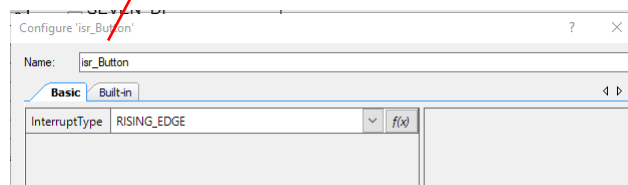


Step 3 – Implementing the ISR in code

- Again, use exactly the same name as before
- Use the ISR1 or ISR2 macro as required

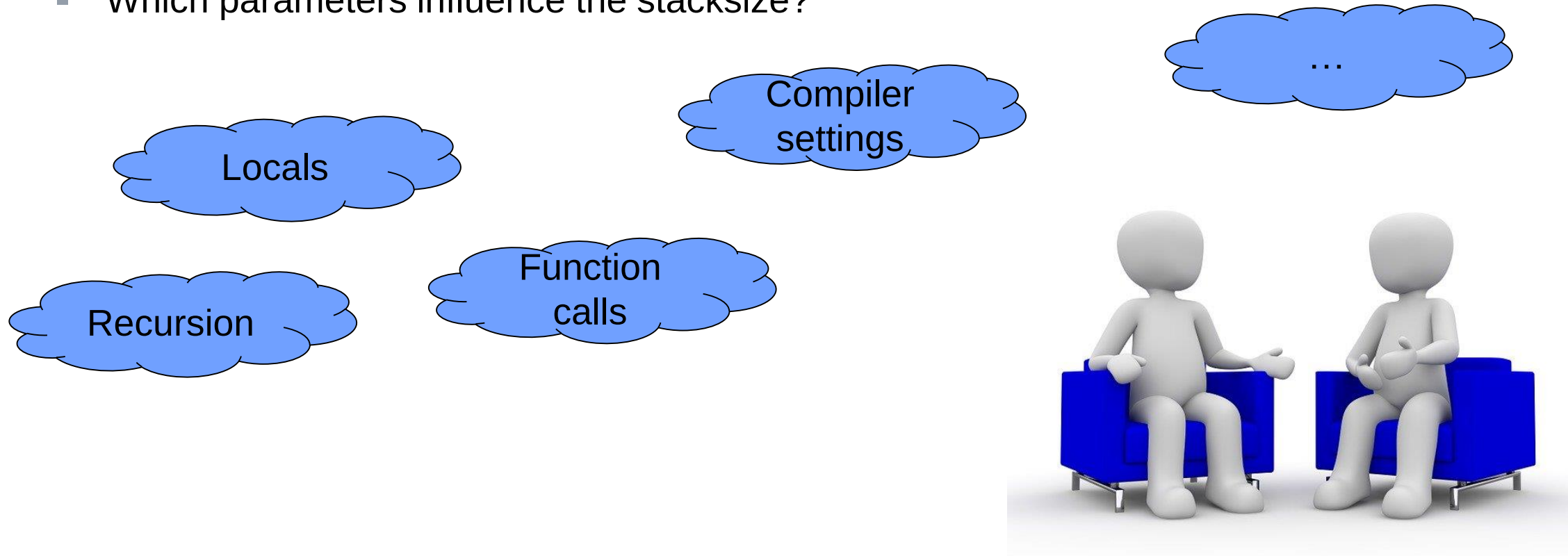


```
130  
131 ISR2 (ISR_Button)  
132 {  
133     SetEvent (tsk_sevenSet, ev_reset);  
134 }
```



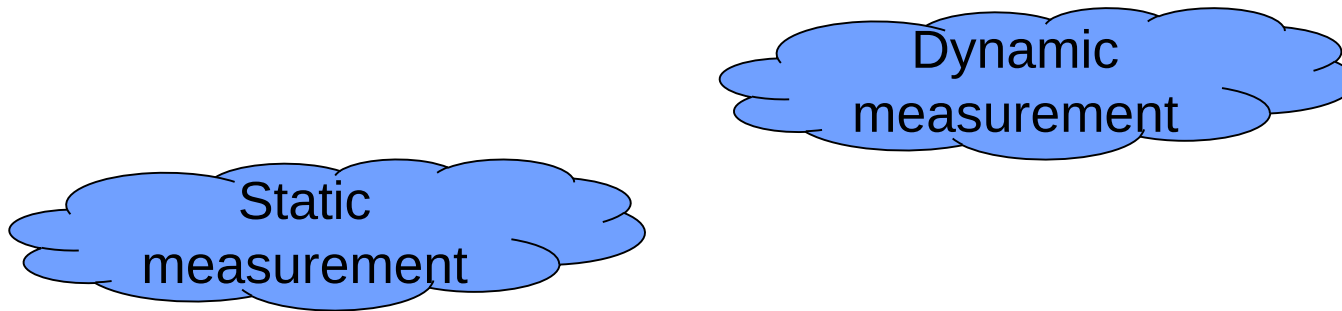
Estimating Stack Size

- In a RTOS, tasks often have their private stack (in OSEK, only extended)
- Which parameters influence the stacksize?



How can we measure stacksize?

- How can we know, if we have provided sufficient stack?



Starting point for Erika: 200bytes (required for
WaitEvent,GetEvent,ClearEvent)
1500bytes if we want to use the comfortable LOG function (sprintf)



Wrap-Up

- OSEK / Erika API
- Setting up a professional workspace
- Task Design Patterns
- Interrupts
- Stack size